

L05: Animation and Sound

Chris Carvelli

Agenda

1. Linear interpolation and splines
2. Types of animation
3. Sound

Final Project requirements

Hard requirements

- must be done in groups of 2/3 people
- must be a C++ project using SDL

Types of projects

- a C++ game made with SDL
- a tool for game development (either for design or runtime)
- an improved implementation of one of the systems that we looked in the course
 - better scene structure
 - a custom collision system
 - an animation/particle system high-level API
- Check the course ILO's on learnIt for guidance

Formal requirements will be on learnIt

Mandatory assignments and re-exams

- You have 3 attempts to pass the exam:
 - Main exam (January)
 - Re-exam (Feb./Mar.)
 - Extra (next year)
- The extra attempt will be done with next year material/exercises.

Game Programming (Autumn 2024)

[Course](#) [Settings](#) [Participants](#) [Grades](#) [Reports](#) [More](#) ▾

▾ About this course

Main book: "Game Engine Architecture, Third Edition" by Jason Gregory (ISBN: 9781138035454)



News



Ordinary exam assignment



Available from 16 December 2024

Two important links

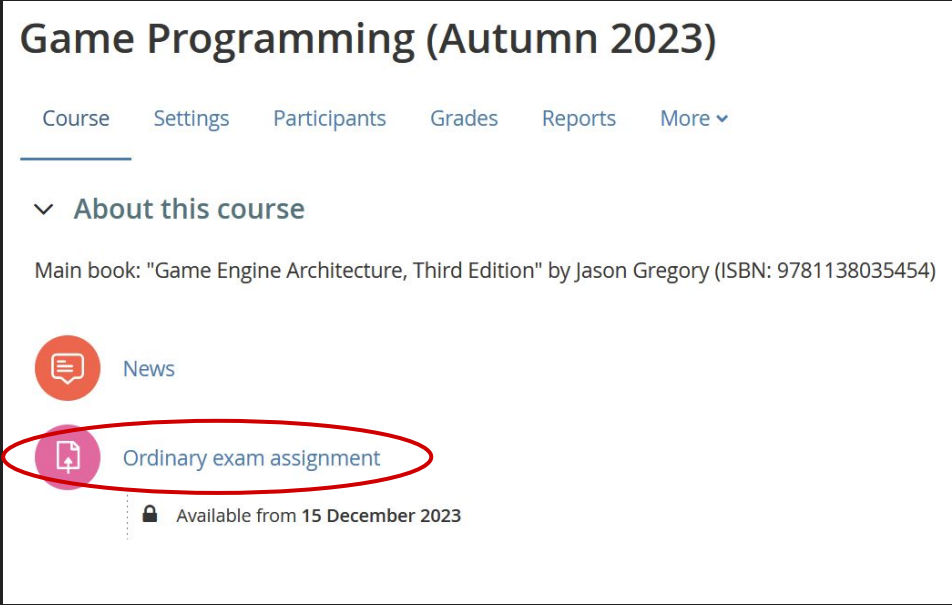
- Formal Exam Project Requirements
- Exam Schedule

*This activity lists the mandatory literature for the examination.
The examination syllabus is published before the last lecture or earlier.*

-  Library for simple for console rendering and input utils (`ITUGames.h`)
-  SREP dependencies 2024
-  Formal Exam Project Requirements
-  Exam schedule (room: 3A54)
Hidden from students
-  Course Evaluation Survey (Autumn 2024)

Handing in

- Submission must contain:
 - Full source code
 - Executable
 - Report
- Deadline: January 3rd at 14:00
- Late submissions are not accepted!



The screenshot shows a course page for "Game Programming (Autumn 2023)". At the top, there are navigation links: "Course", "Settings", "Participants", "Grades", "Reports", and "More". Below these links is a section titled "About this course" with a downward arrow. Under this section, it states: "Main book: 'Game Engine Architecture, Third Edition' by Jason Gregory (ISBN: 9781138035454)". Below the book information, there is a "News" section with a red speech bubble icon. Underneath "News", there is a pink circular icon with a document and an upward arrow, next to the text "Ordinary exam assignment". This entire item is circled in red. Below this, there is a lock icon and the text "Available from 15 December 2023".

Game Programming (Autumn 2023)

[Course](#) [Settings](#) [Participants](#) [Grades](#) [Reports](#) [More](#) ▼

▼ **About this course**

Main book: "Game Engine Architecture, Third Edition" by Jason Gregory (ISBN: 9781138035454)

 [News](#)

 [Ordinary exam assignment](#)

 Available from 15 December 2023

Exam

- Be early
- Be ready for possible delays
- Group presentation (5 min)
- Individually examination
 - Questions about your game
(based on report / implementation)
 - Questions in syllabus
(Short concise answers)
- Grading
- Total time per student: 20 minutes

5 Min. Group presentation

15 Min. Individual examination

15 Min. Individual examination

15 Min. Individual examination

Grading

- Project hand-in 50%
- Exam performance 50%
- Based on how well you meet the learning objectives

✓ W2 - C++ language foundations

Learning objectives

- * Game Engines architectural styles
- * GameObject-Component architecture
- * Game Loops design patterns
- * Get familiar with additional C++ features
 - * Preprocessor macros
 - * Source files, object files, executables and libraries
 - * Standard Template Library (STL)
 - * Classes (and relative concepts like encapsulation and inheritance)

Required readings

- * None (but be comfortable with last lecture's C++ concepts)

Additional Resources

- * C++ Language tutorial (<http://www.cplusplus.com/doc/tutorial/>) except pointers and dynamic memory.
- * C++ cheat sheet (<https://github.com/mortennobel/cpp-cheatsheet>)

Exam schedule

- Check most recent version on LearnIt (in January)
- Write ASAP if you have specific scheduling needs (or better, find another group to switch with - possibly same size as yours)
- Exam dates cannot be changed, without exceptions
- BUT, exam can be done online in special cases

Previous projects

Steamphonk

- 2D
- Tilemap (integration with 3rd party tool)
- Used some concepts from the exercises, but improved and adapted



Scap Hunters

- 2D
- Procedurally generated levels
- Nice extra touches (audio and particles)



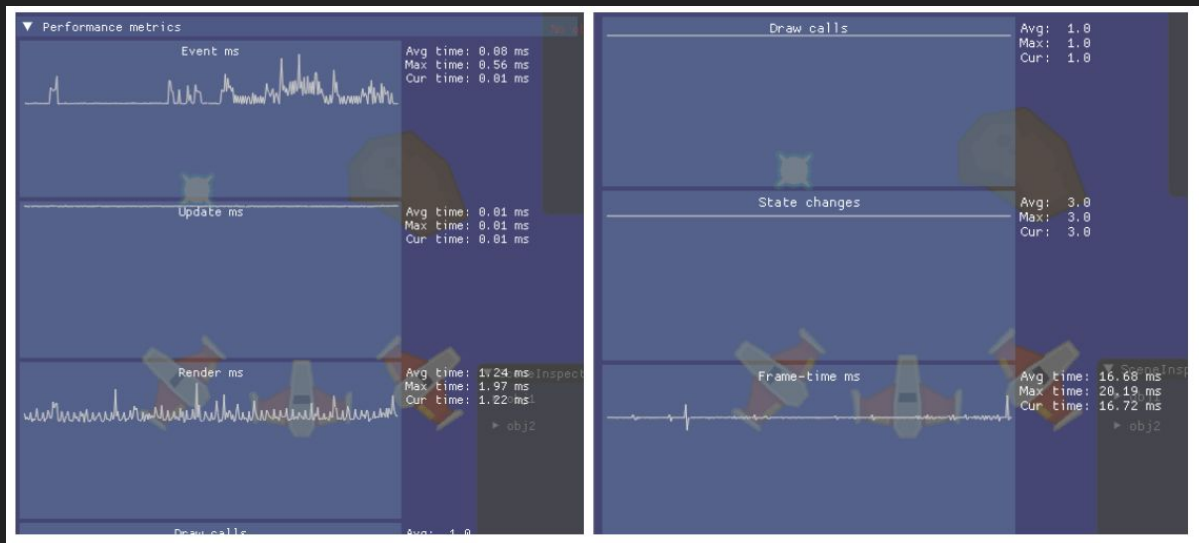
Golf Game

- 3D, bullet integration
- Multiple levels
- Tile set from Kenney



World Editor

- engine (extension) example
- reworked the given scene graph is represented and serialized
- added GUI editing to object and components



Questions about the exam?

Early feedback results



Exercise 04 review

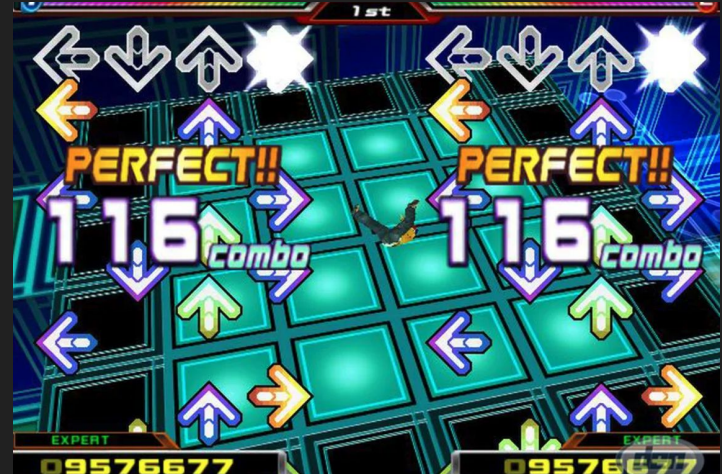


L05 - Animation and Sound

Problem statement: How do we...



- ...add audio to our games?
- ...manipulate audio to leverage game's multimodality and interactivity?



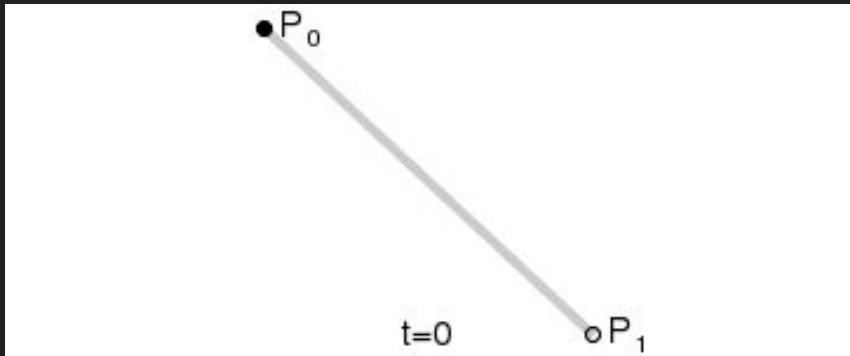
- ...use the math that we know to emulate motion?
- ...smooth continuous motion with proper control?

Agenda

1. Linear interpolation
2. Types of animation
3. Sound

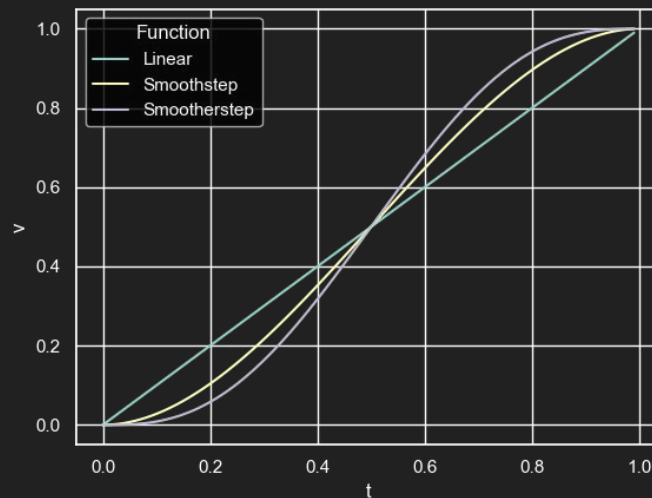
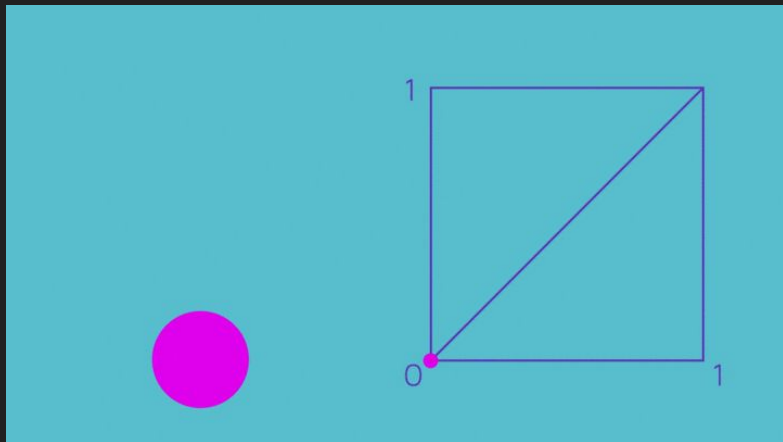
Linear Interpolation

- Finding intermediate value between two values (P0 and P1)
- Using some fraction, usually time (t)
- $\text{pos} = p_0 * (1.0 - t) + p_1 * t$
- `glm::mix(p0, p1, t)`
- Movement in constant velocity (relative to t)



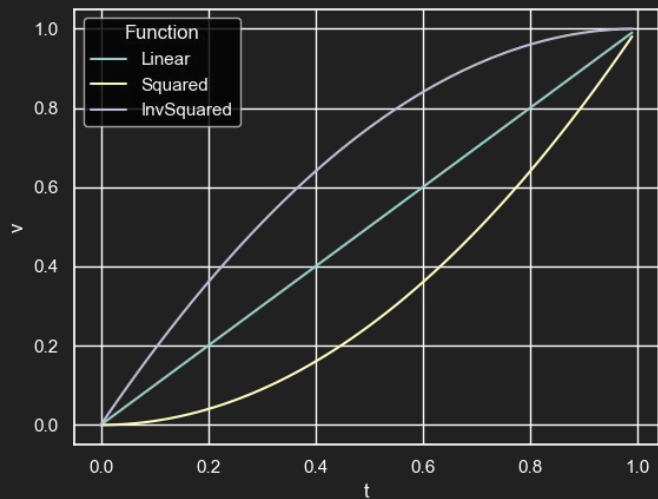
Interpolation using easing functions

- Using easing functions allows motion to accelerate / decelerate
- Can be used to modify t input for `lerp()`

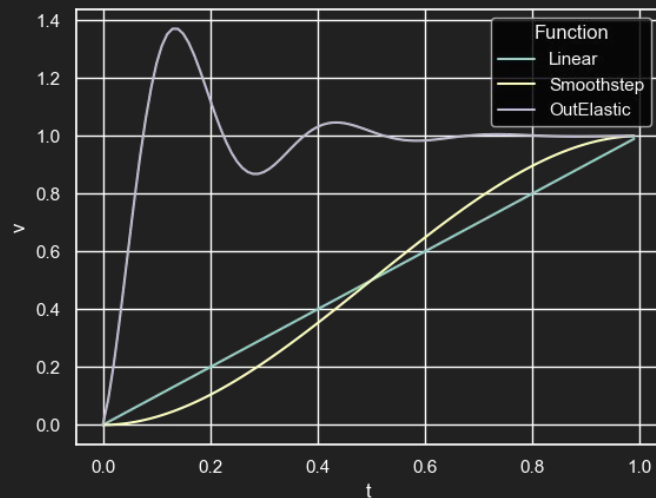


```
float t = 0.5f;  
float v = smoothstep(t);  
vec2f pos = lerp(from, to, v);
```

Other easing functions



```
// squared / in-quad  
float v = t * t;  
  
// inv. squared / out-quad  
float v = 1 - (1 - t) * (1 - t);
```



```
// smoothstep  
t = SDL_clamp(t, 0, 1);  
float v = t * t * (3.0f - 2.0f * t);  
  
// out-elastic  
float c4 = (2.0f * PI) / 3.0f;  
float v = pow(2, -10 * t) * sin((t * 10 - 0.75) * c4) + 1;
```

Easing function cheat sheet

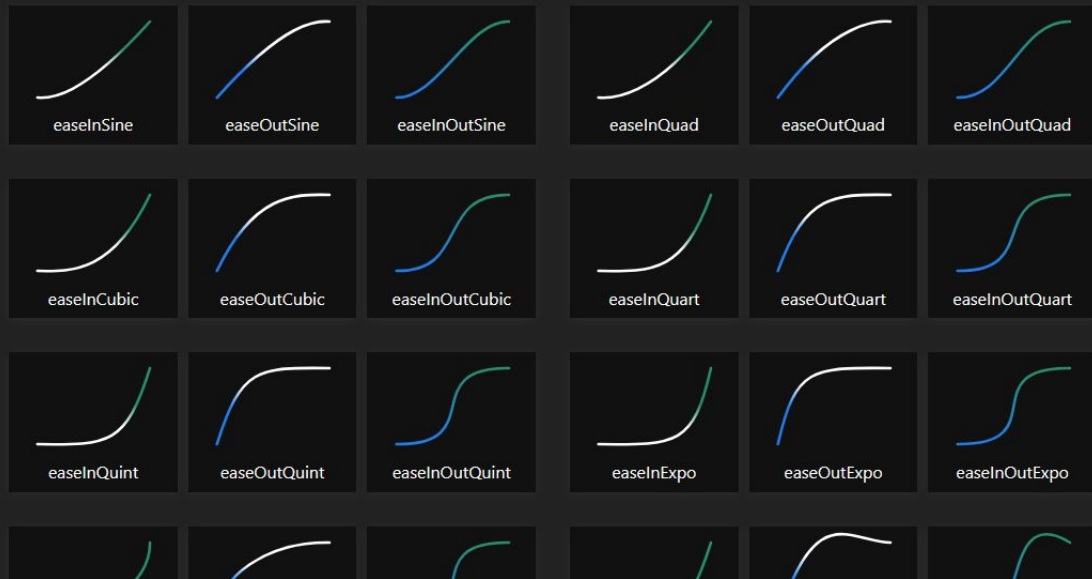
Easing functions specify the rate of change of a parameter over time.

Objects in real life don't just start and stop instantly, and almost never move at a constant speed. When we open a drawer, we first move it quickly, and slow it down as it comes out. Drop something on the floor, and it will first accelerate downwards, and then bounce back up after hitting the floor.

This page helps you choose the right easing function.

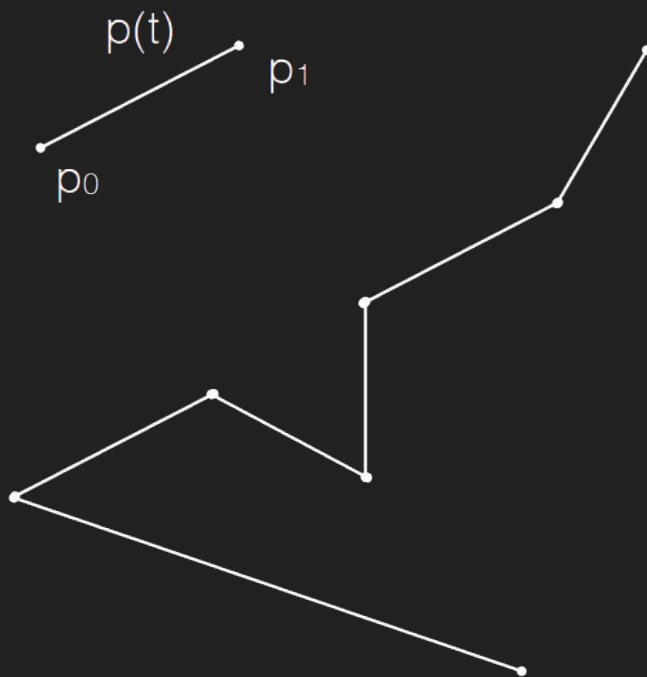
 Open Source

Help translate
site to your language



Moving between two points

- Given two or more points, how can we move between them?
- Linear interpolation:
 $p(t) = p_n + t(p_{n+1} - p_0)$
- This causes very jagged movements.
- We can do better



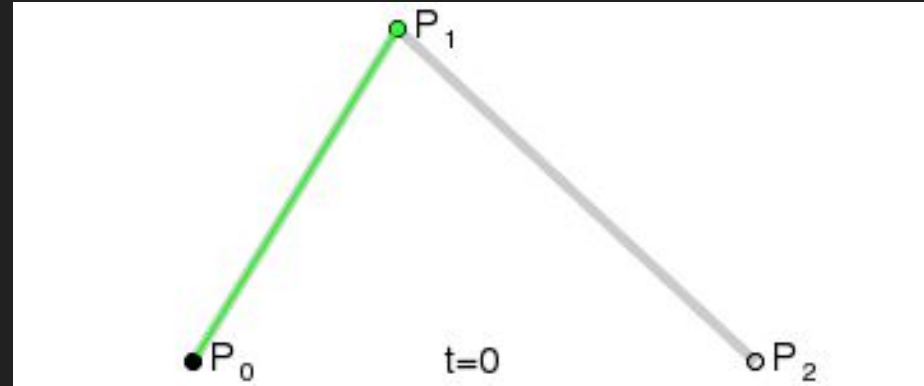
Origin of splines

- Historically to draw curves, especially for shipbuilding, splines were used.
- Splines were usually long, thin, flexible strips of wood, plastic, or metal.
- The elasticity of the spline material and the position of the control points forms the smooth curve.



De Casteljau Algorithm

- What if we had two lines (3 points)
- Find two points by
 - $t_0 = \text{lerp}(p_0, p_1, t)$
 - $t_1 = \text{lerp}(p_1, p_2, t)$
- Finally interpolate between these two points
 - $p(t) = \text{lerp}(t_0, t_1, t)$



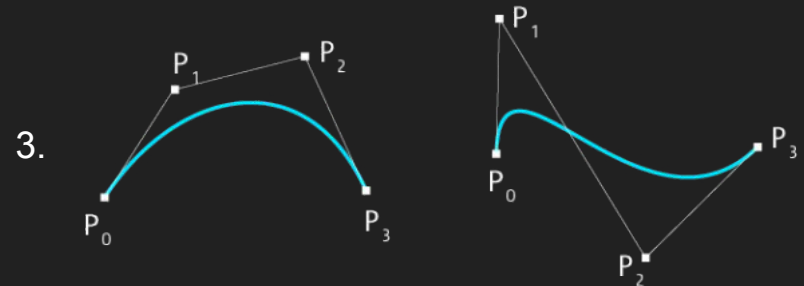
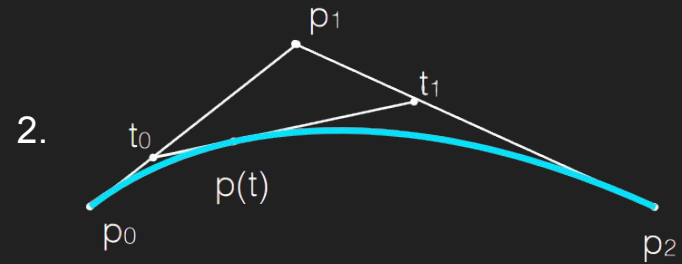
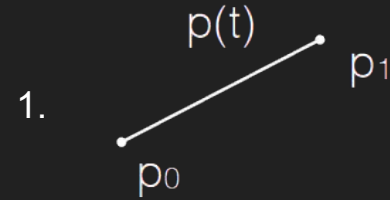
Bézier curves

1. $p = \text{lerp}(t_0, t_1, t)$ Linear

2. $t_0 = \text{lerp}(p_0, p_1, t)$ Quadratic
 $t_1 = \text{lerp}(p_1, p_2, t)$
 $p = \text{lerp}(t_0, t_1, t)$

3. $t_0 = \text{lerp}(p_0, p_1, t)$ Cubic
 $t_1 = \text{lerp}(p_1, p_2, t)$
 $t_2 = \text{lerp}(p_2, p_3, t)$
 $t_3 = \text{lerp}(p_2, p_3, t)$
 $t_4 = \text{lerp}(p_2, p_3, t)$
 $p = \text{lerp}(t_3, t_4, t)$

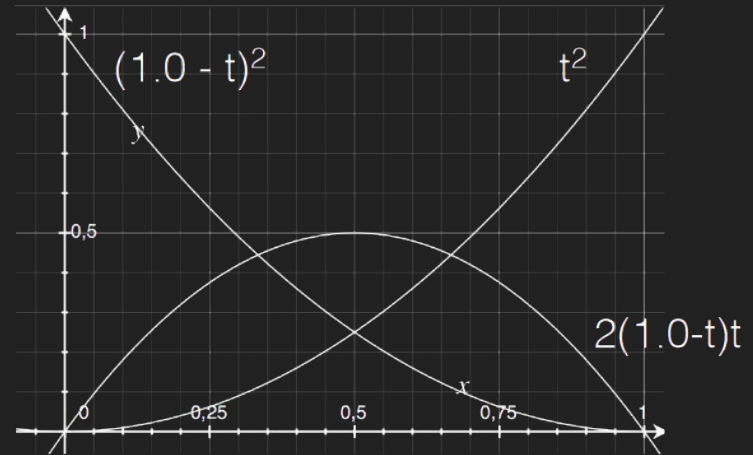
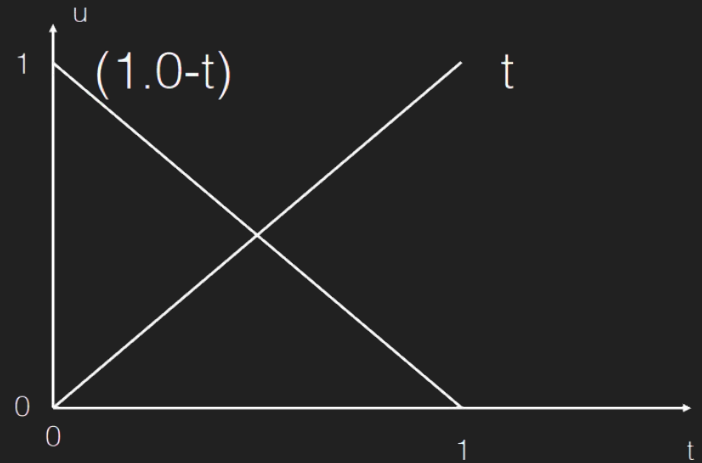
Readable (sort of), but verbose
and not efficient.



Basis curves

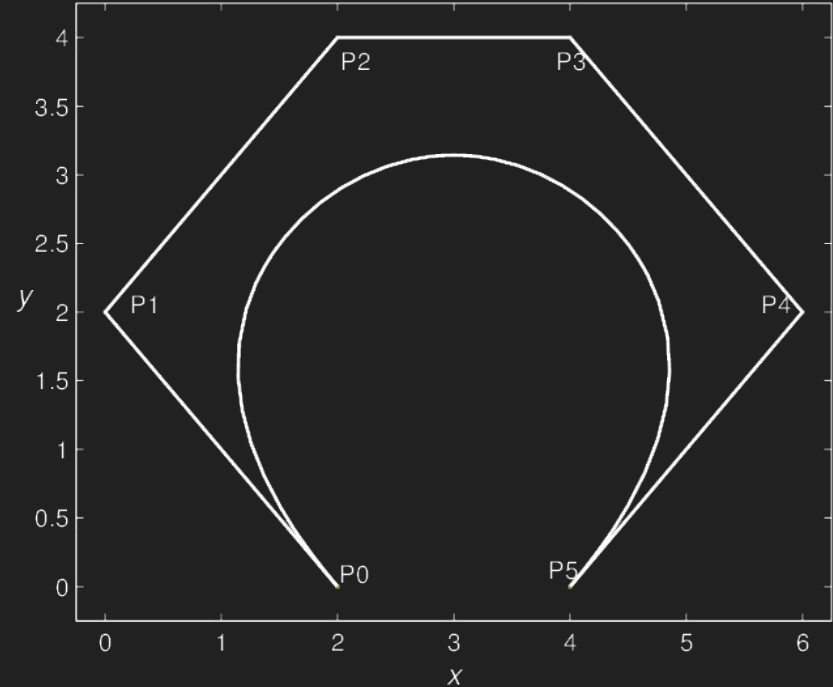
- Linear bézier $(1-t)p_0 + (t)p_1$
 - $b_0(t) = (1 - t)$
 - $b_1(t) = t$
- Quadratic bézier
 - $b_0(t) = (1 - t)^2$
 - $b_1(t) = 2(1 - t)t$
 - $b_2(t) = t^2$
- Cubic bézier
 - $b_0(t) = (1 - t)^3$
 - $b_1(t) = 3(1 - t)^2t$
 - $b_2(t) = 3(1 - t)t^2$
 - $b_3(t) = t^3$

More compact and efficient.



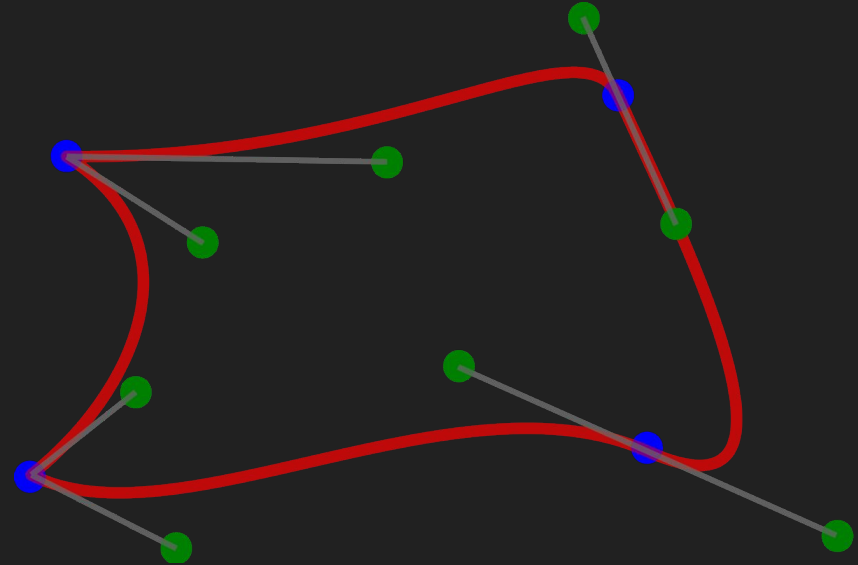
Problems with Bézier curves

- Béziers do not scale well in the number of points
 - Performance: more and more basis functions, more and more complex
 - Control: changing a control point will change the entire curve!



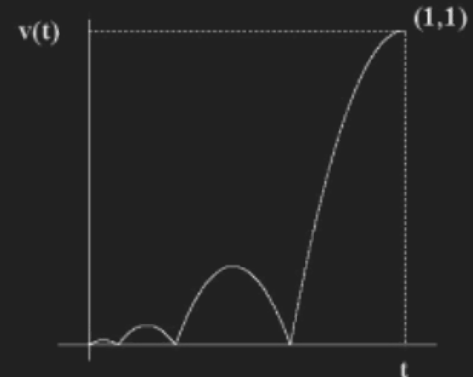
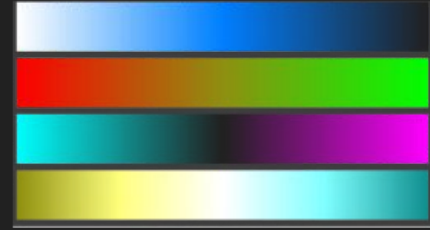
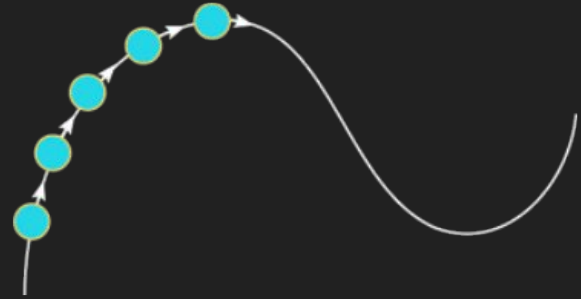
Splines

- A spline is a parametric piecewise curve (find point from parameter t)
- Based on
 - control points
 - (optional) tangent vectors
- Desirable properties:
 - Can easily compute first and second derivative (velocity and acceleration)
 - Control points affect only a limited section of the spline
- Built concatenating together multiple basis curves, like béziers



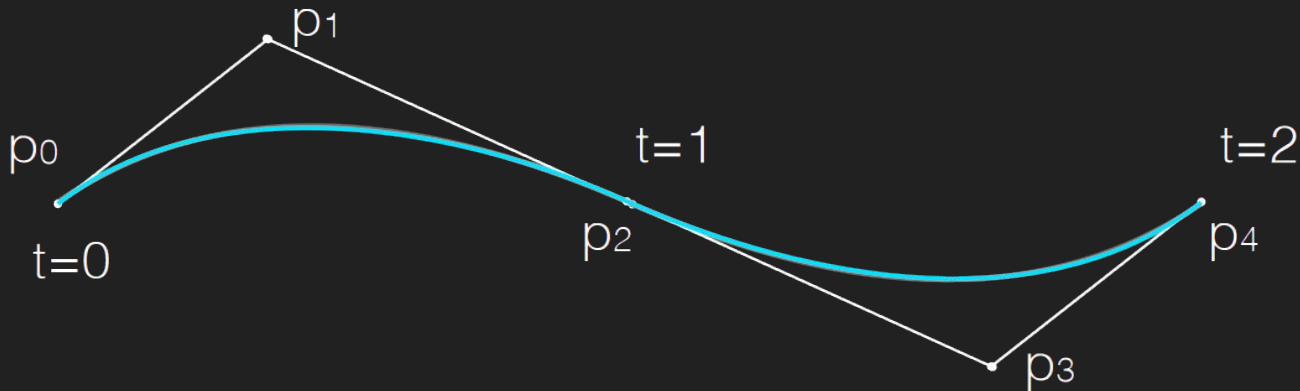
Splines use cases

- Interpolate between a number of values (such as floats or vectors)
- Examples
 - Moving objects along a curve
 - Mixing colors
 - Tweaking component properties over time
 - Sounds



How to evaluate a spline

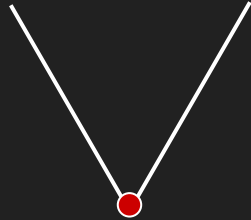
- Example find point of $t = 1.5$
- Two Bézier curve segments $[p_0, p_1, p_2]$, $[p_2, p_3, p_4]$
- 1.5 is in the second segment
- Normalize $t=1.5$ to the segment ($t_n = (1.5 - 1) / (2-1) = 0.5$)
- Evaluate bezier curve using $[p_2, p_3, p_4]$ and 0.5



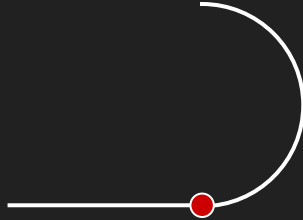
Continuity

Used to describe the smoothness of curves

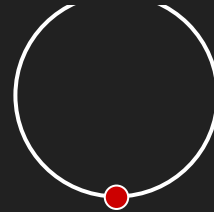
- C^0 = Continuous position
- C^1 = Continuous velocity
- C^2 = Continuous acceleration



C^0 continuity



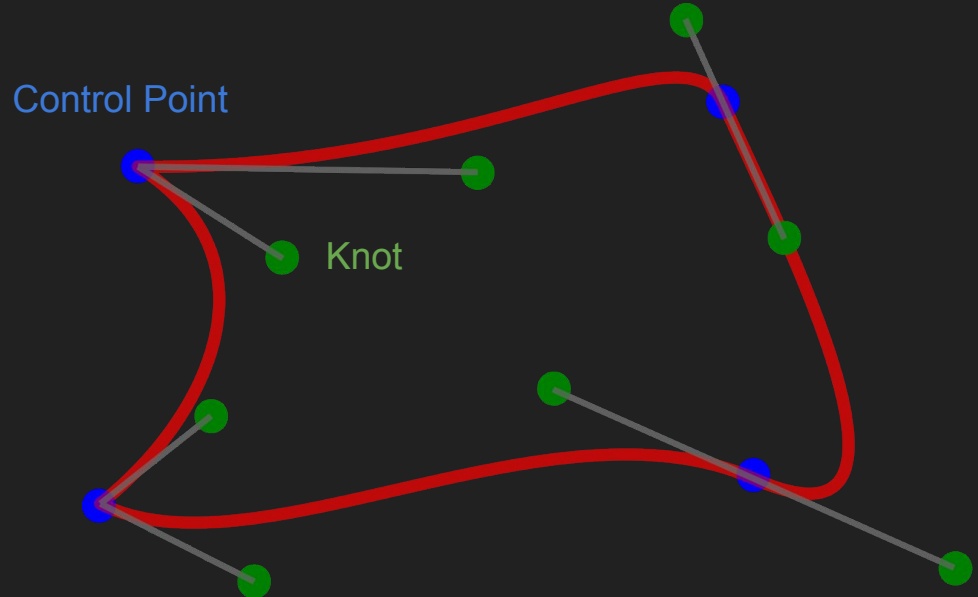
C^1 continuity



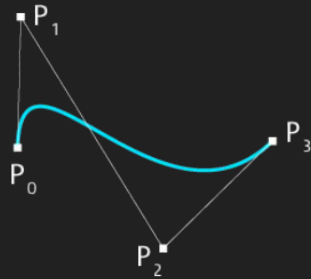
C^2 continuity

Control points

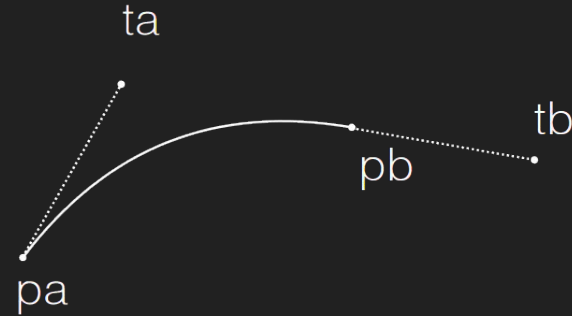
- To get splines with arbitrary kind of continuity, we can cheat using
- We group the points of our curve in control points
 - The main control point is the point that the curve has to pass through ("fit")
 - The two external points, or knots, are an indication of what shape the curve can take
- Imposing constraints on the knots we can enforce continuity.
 - C1 => knots of the same control point must lie on the same line
 - C2 => C1, and knots must be at the same distance from the relative control point
 - And more! Check you favorite digital drawing tool for inspiration



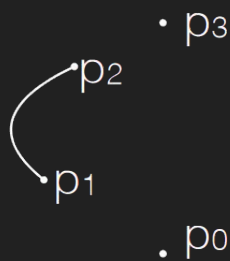
Other types of spline curves



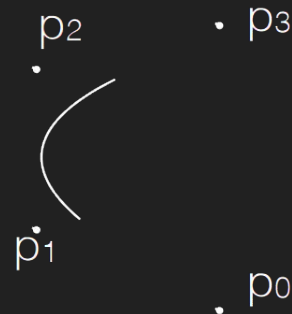
Cubic Bézier



Piecewise Hermite



Catmull-Rom



Uniform cubic B-Spline

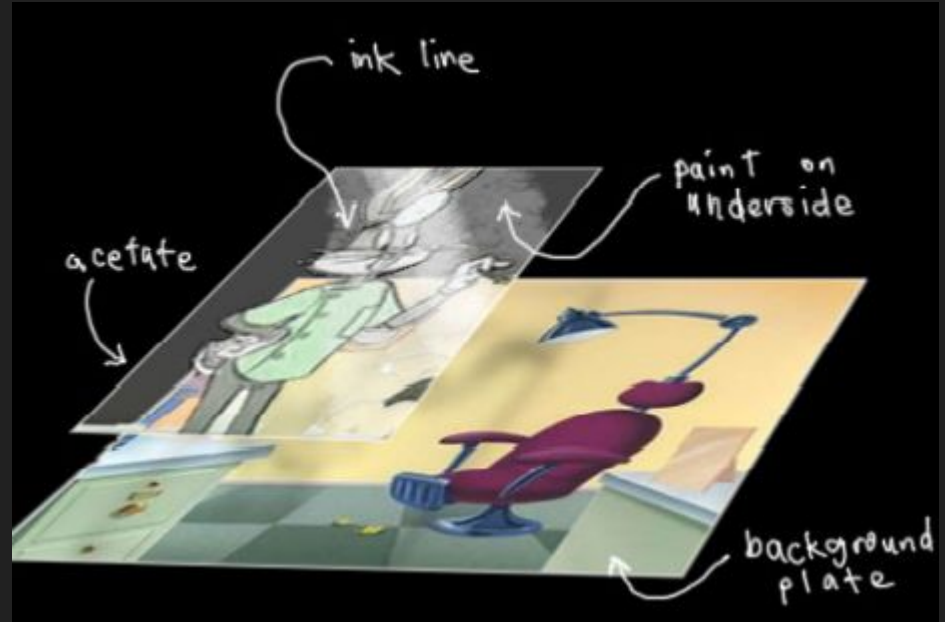
<https://www.youtube.com/watch?v=o9RK6O2kOKo>

Agenda

1. Linear interpolation
2. Types of animation
3. Sound

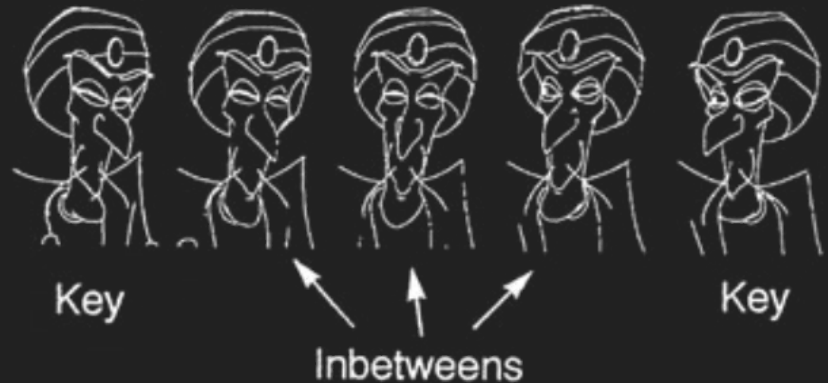
Classic Cel Animation

- Hand drawn animations made famous by Disney for animated movies
- 24 fps (1440 frames per minute)
- Cel stands for celluloid which is a clear sheet which images drawn on them
- Fixed background



Keyframes in cel animation

- Very time consuming drawing frames for animated movies
- Lead artists would draw only the most important frames in a sequence called keyframes
- Junior artists would draw the frames in-between (the process also called tweening)

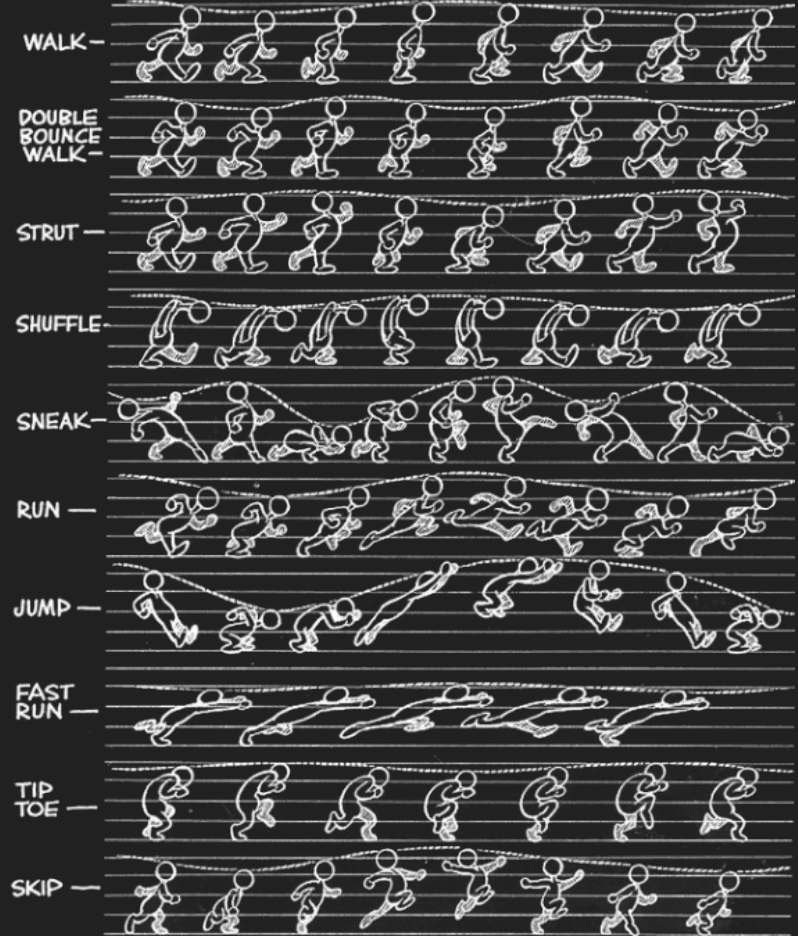


Animation cycles

- Looping sequence of frames
- Example: various ways of walking

MOVEMENTS OF THE TWO LEGGED FIGURE

HERE IS A COMPARISON OF THE VARIOUS TWO LEGGED FORWARD MOVEMENT CYCLES--I HAVE DRAWN ONE HALF OF EACH CYCLE BELOW--REVERSE HANDS + FEET FOR THE OTHER HALF--THESE CYCLES CAN BE USED AS 'REPEATS'--(THAT IS THE DRAWINGS MAY BE REPEATED OVER + OVER IF THE FIGURE REMAINS CENTERED ON THE SCREEN AND THE BACKGROUND MOVES.





Cel/sprite animation



Rigid hierarchical animation



Per-vertex animation

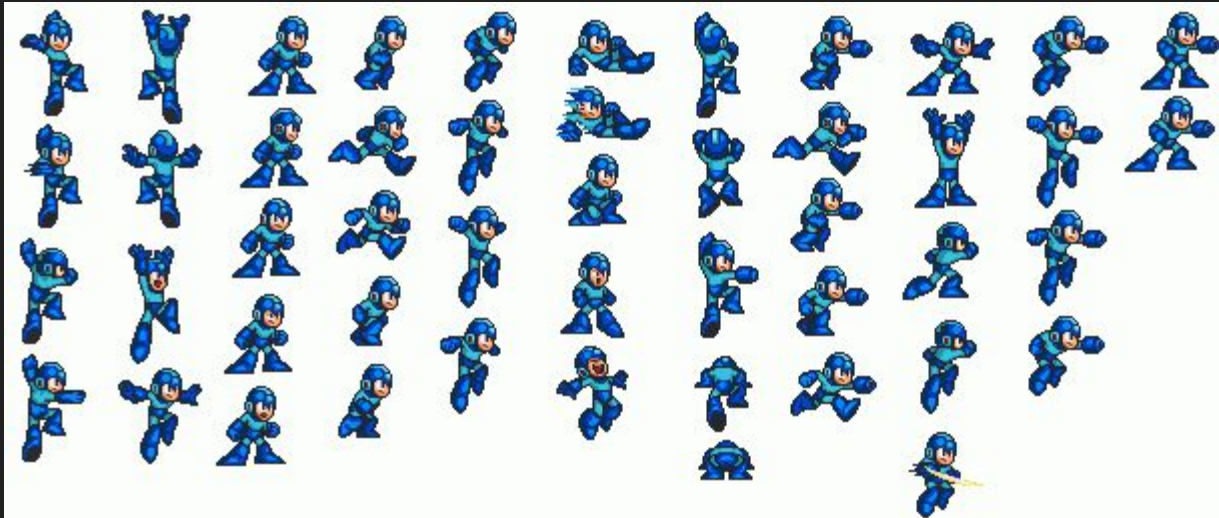


Skinned animation

Sprite animation

- Sprites animations is a raster based technique inspired by cel animation.
 - Usually stored in a sprite sheet (for optimisation)
 - Optionally can be looped.
- 

Spritesheet from megaman



Sprite animation for 3D games

- Distant objects or GUI (billboarding)
- Pseudo 3D games (Quake)
- Artistic choice (TOEM)
- UI



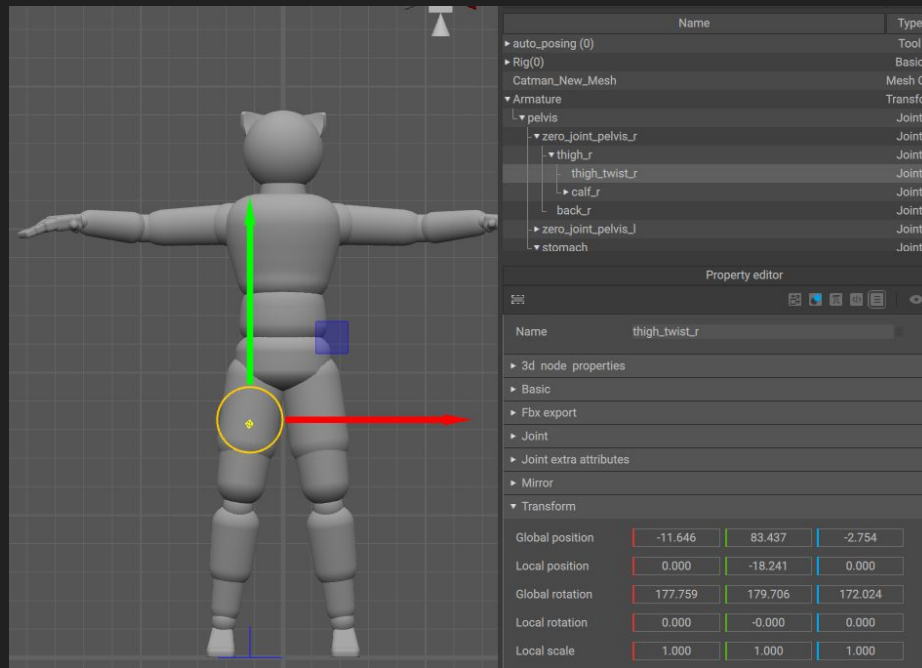
Rigid hierarchical animation

- Divide character into non-deformable parts in a scene hierarchy
- Animate the transforms of the hierarchy
- Works well for robots and similar rigid objects



Rigid hierarchical animation

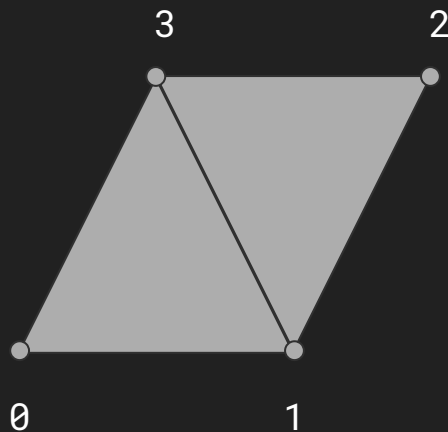
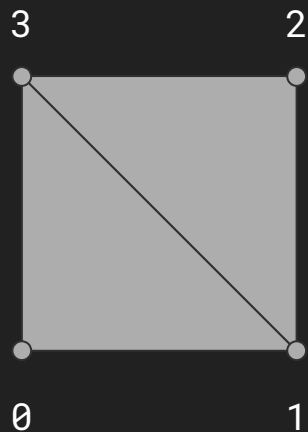
- Keyframes are defined with a transform matrix, usually called SQT (Scale, Quaternion rotation, Translate)
- In-betweens are computing interpolating two consecutive keyframes SQT transformations.



Per-vertex animation

- Simple way to do non-rigid animation
- Define keyframes by having multiple positions for each vertex
- In-between frames can be found using linear interpolation

i	pos keyframe 0	pos keyframe 1
0	(0, 0, 0)	(0, 0, 0)
1	(1, 0, 0)	(1, 0, 0)
2	(1, 1, 0)	(0.5, 1, 0)
3	(1, 0, 0)	(1.5, 1, 0)



Per-vertex animation problems

- Takes a lot of data
- Difficult to author
- Almost impossible to reuse



Per-vertex animation other usages

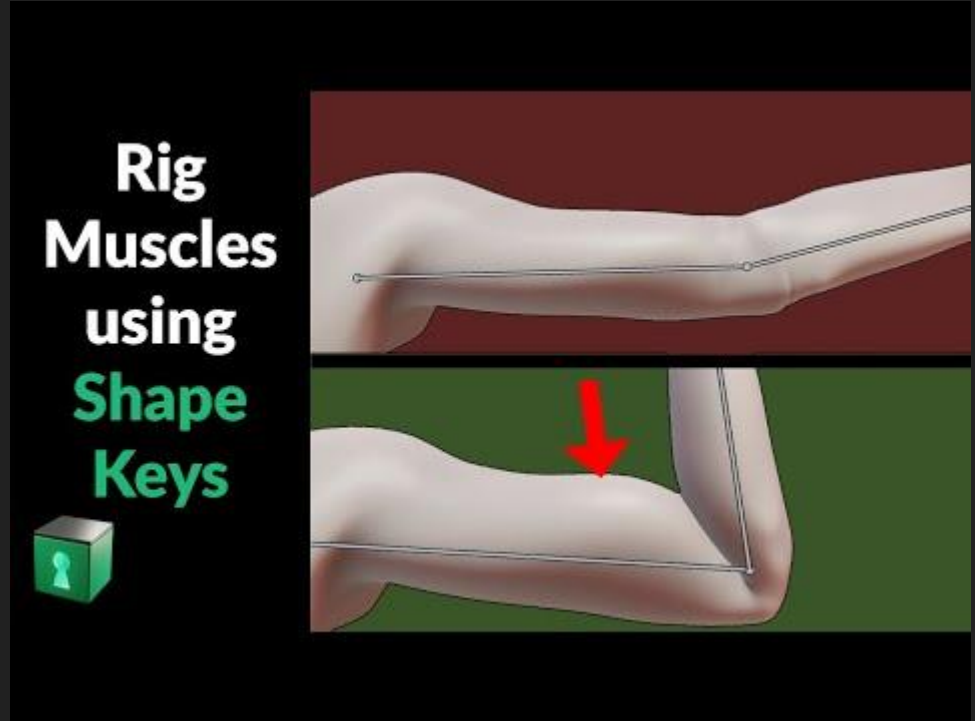
- Animated shaders
- Improvement of other kind of animation
- Cool game effects



<https://80.lv/articles/creating-next-gen-grass-in-ue4/>
<https://www.artstation.com/artwork/Vy0OAn>

Per-vertex animation other usages

- Animated shaders
- Improvement of other kind of animation
- Cool game effects



Per-vertex animation other usages

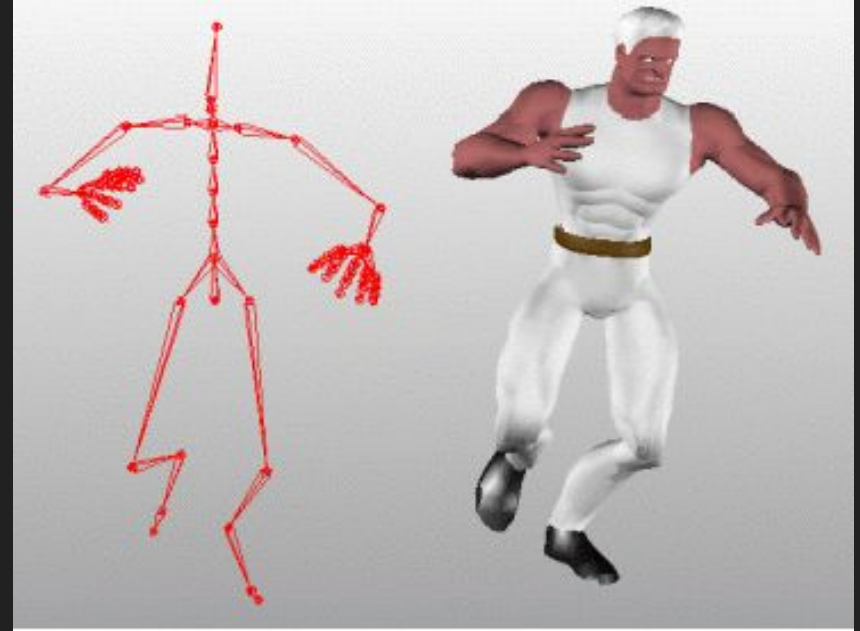
- Animated shaders
- Improvement of other kind of animation
- Cool game effects



<https://www.patreon.com/posts/22650450>

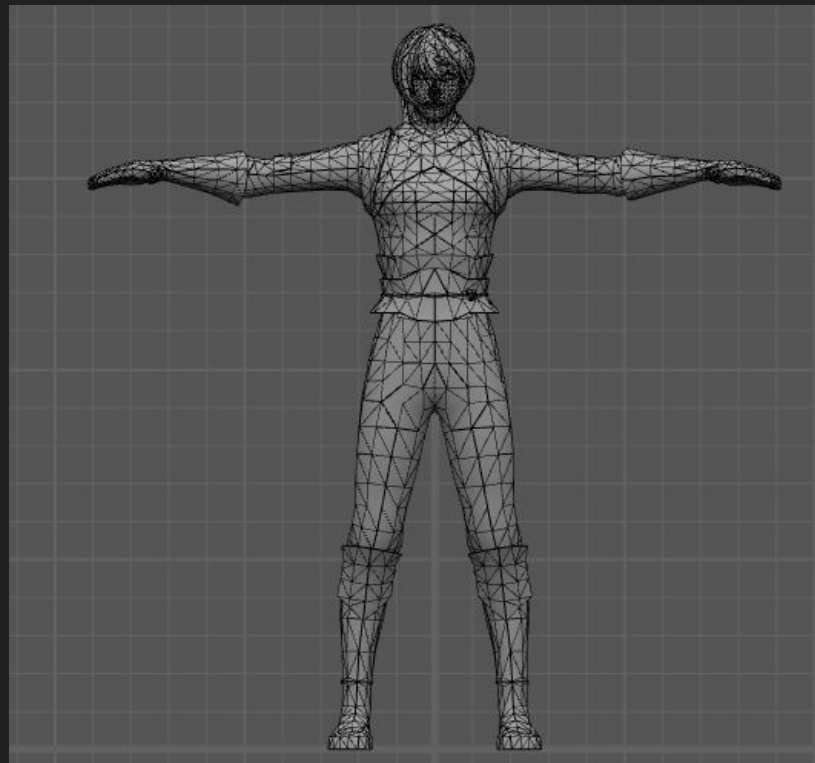
Skinned animation

- Can be seen as a combination of rigid and vertex animations
- We create a skeleton animated with rigid hierarchical animation.
- Each vertex is mapped to one or more 'bones' in the skeleton.
- We can find the animated position using the transformation of the skeleton bones.



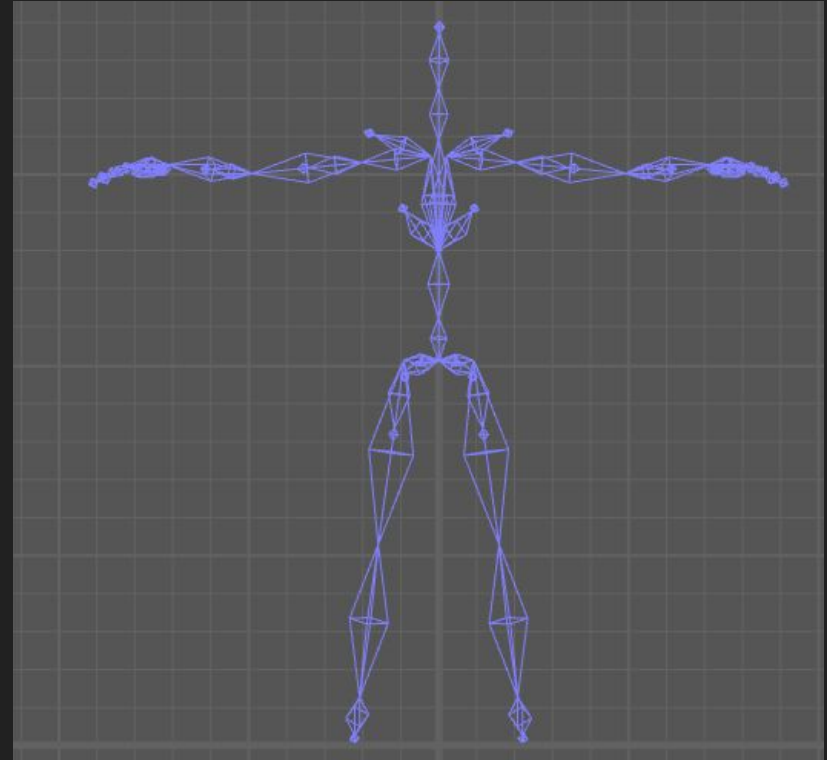
Bind pose

- Aka. reference pose / rest pose / Jesus pose / T-pose
- Arms to the side
- Feet apart
- Making mapping of vertices to bones easy



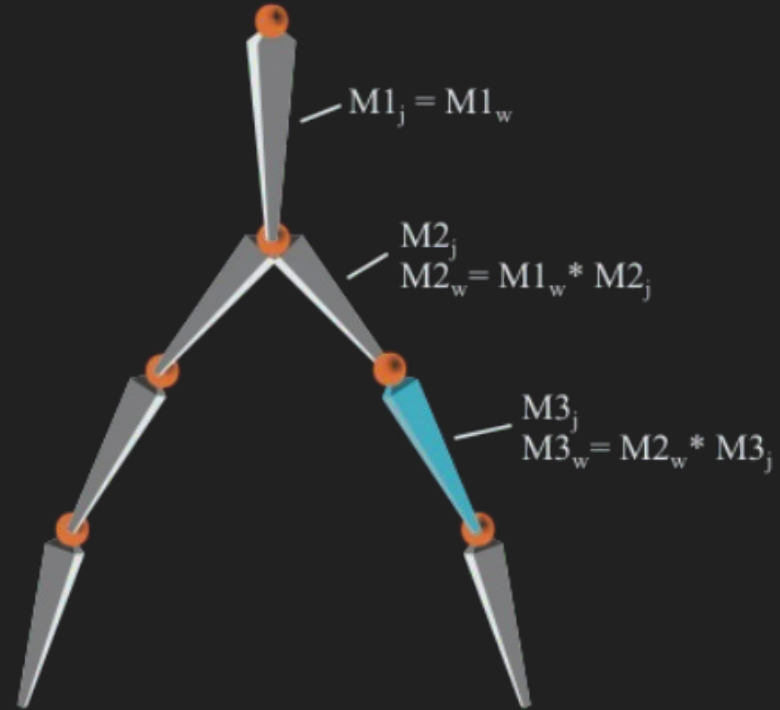
Skeleton hierarchy

- A simplified structure what is being modeled
- Humans are often modeled using same structure
- The hierarchy is put in the model



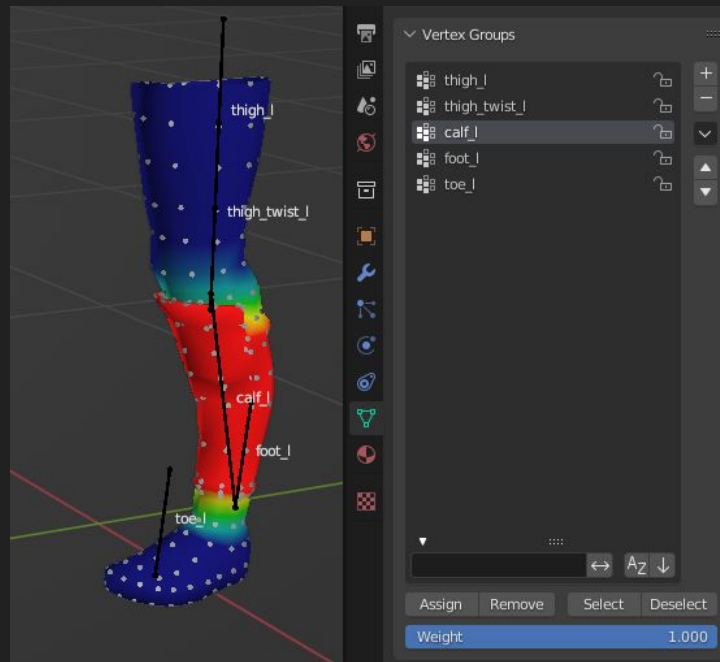
Skeleton hierarchy

- Joins and bones are the same thing (a bone is the space between two joins)
- Each join has
 - Join index
 - SQT transformation
 - Parent index



Rigging models

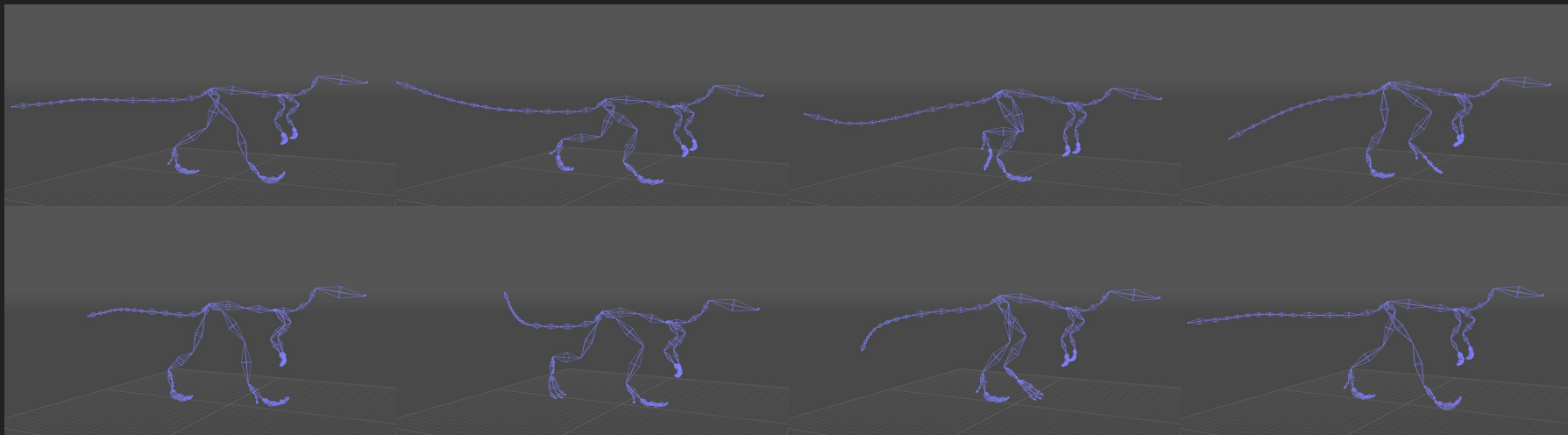
- Mapping each vertex to a number of bones
- For performance reasons, a fixed number of bones (usually, each vertex mapped to 4 bones)
- Each bone has a weight and the sum of all weights must be 1.0



```
// example of a vertex data structure
struct myVertexData {
    glm::vec3 position;
    glm::vec3 normal;
    glm::vec2 uv; // texture coordinates
    int bones[4];
    float boneWeight[4];
};
```

Keyframes

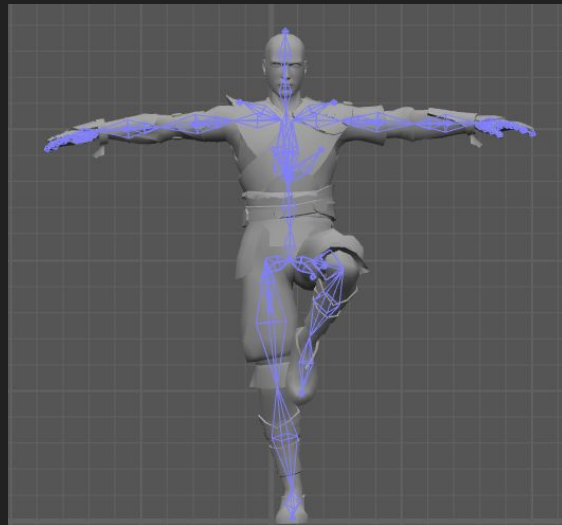
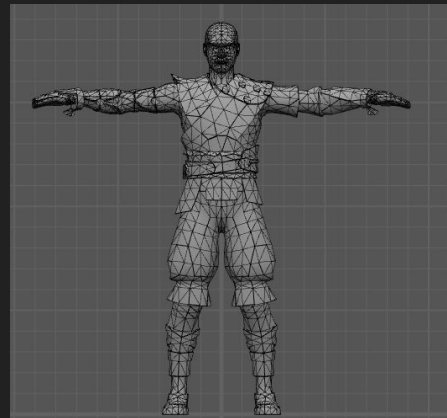
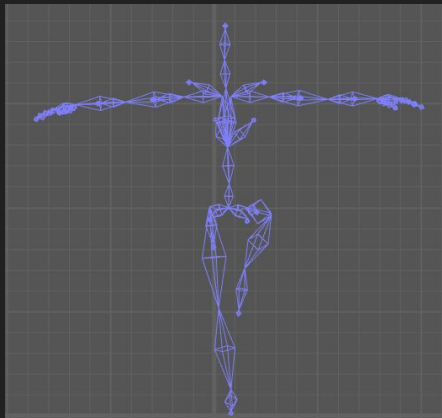
- Defined by modifying the transforms of the skeleton
- Storing the SQT of each joint



Skinned animations

We can now compute the final destination of each vertex position

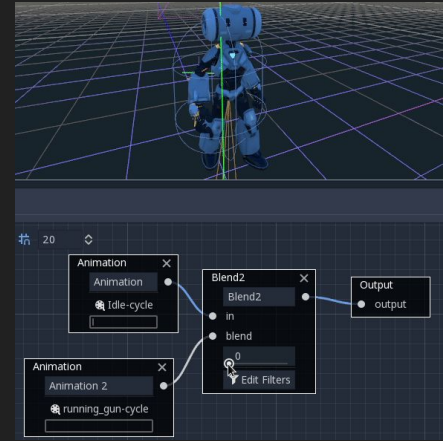
1. Compute skeleton position
2. Compute position of each vertex
 - a. For each associated joint, apply that joint transform to the vertex position
 - b. Average the 4 position obtained



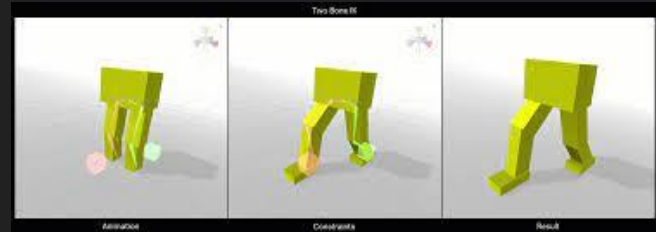
Advanced animation topics



Motion Capture
(digitalization of real performances)



Animation Trees
(smoothly blend between transitions)



Inverse Kinematics
(find position of bones based on targets)

Agenda

1. Linear interpolation
2. Types of animation
3. Sound

Sound in games

Sound types

- Music: Usually plays continuous (loops)
- Audio clips: Sounds played based on game events (jump, collision, etc)
- Procedural Sound: generated based on current game context.

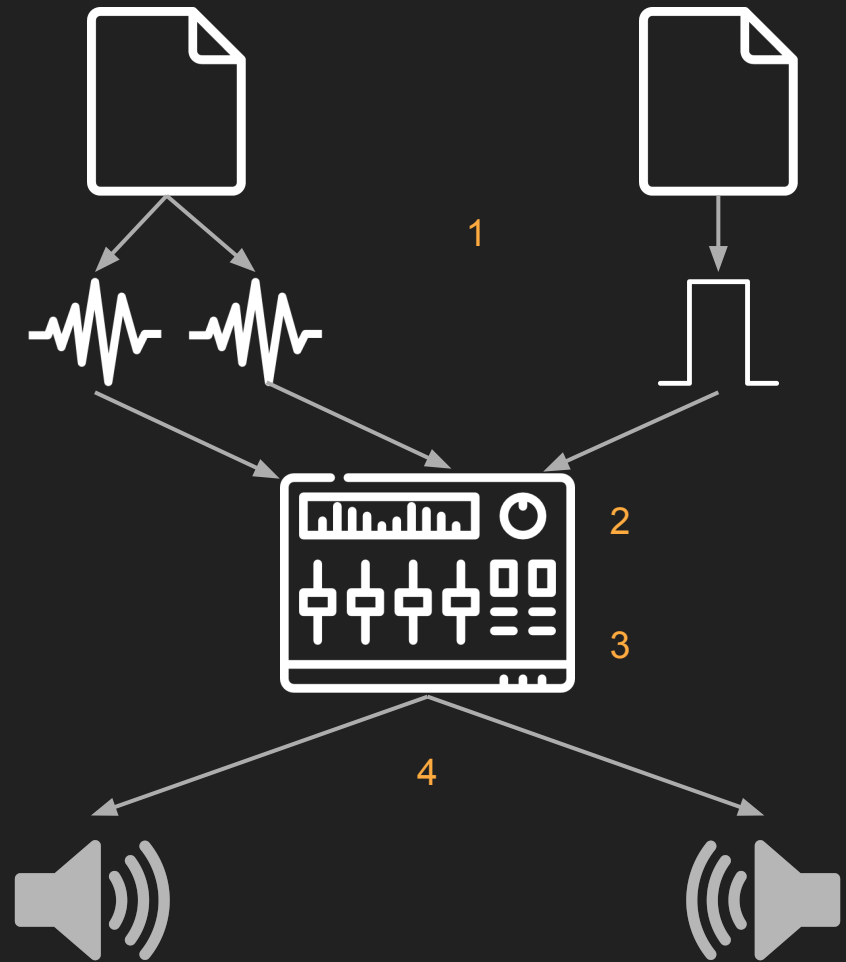
Sound properties

- Sampling rate (frequency/quality)
- Number of channels
 - Mono: single channel
 - Stereo: Left and Right channel
- File extension(wav, mp3)

Playing sound

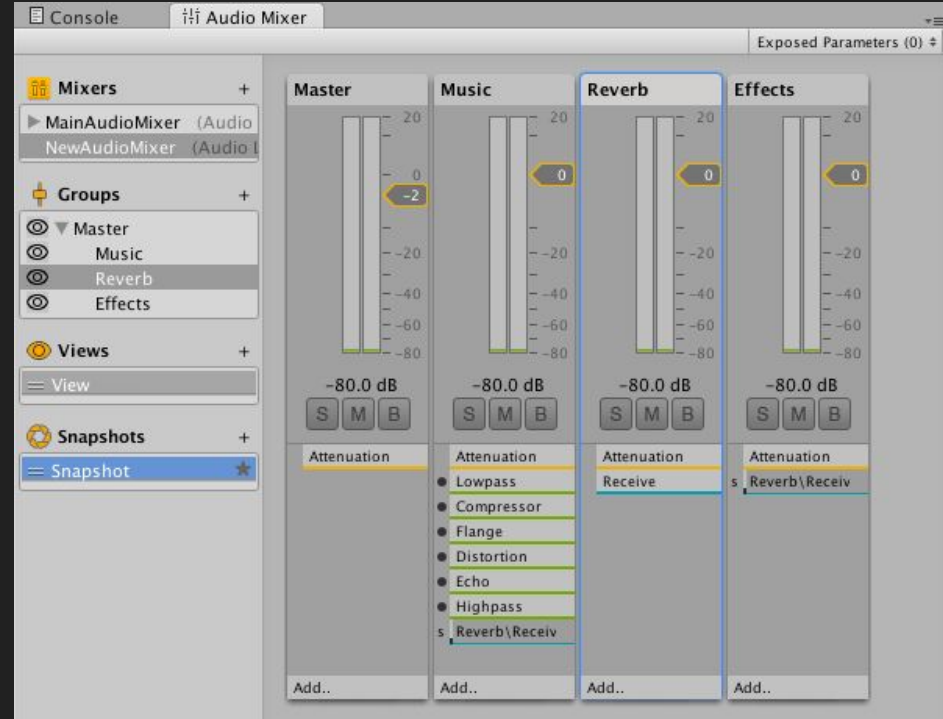
1. Load/uncompress sound assets in the engine
2. Process each sound with effects and filters
3. Mix all of them together
4. Map to physical output

2., 3. and 4. are usually performed by a system called Mixer, like physical device that does the same in concerts, music recording and so on.

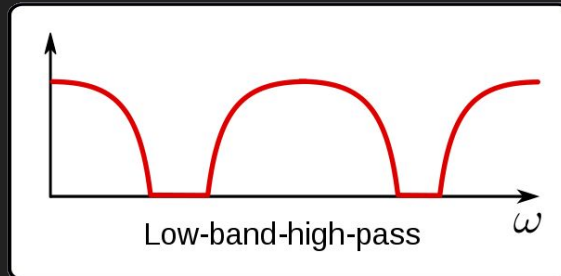
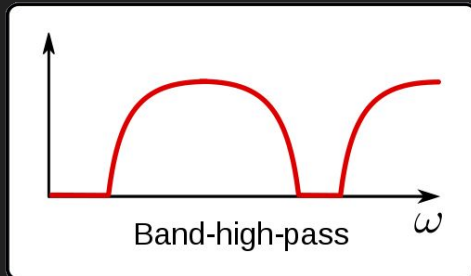
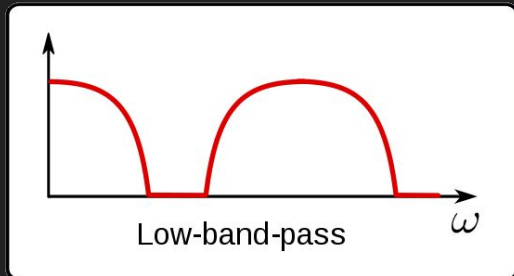
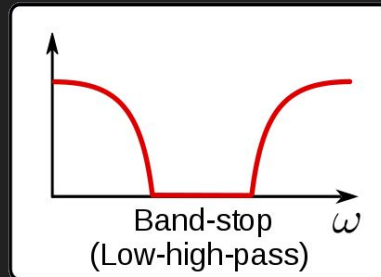
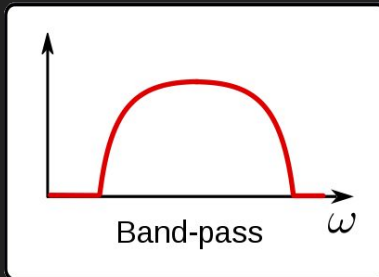
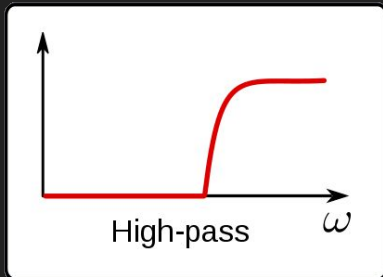
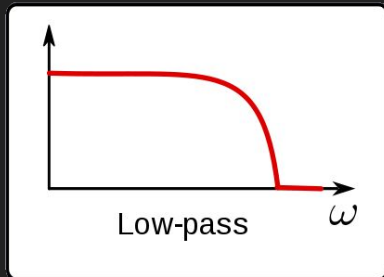


Mixing

- The virtual mixer usually has 8 or more channels.
- Mixing means adjusting how we treat sounds played on different channels.
- Adjusting music volume vs sound effect volume
- Games may have additional layers of mixers, like a user controlled Master Volume, with underlying sliders for sfx, music, voice over, etc.
- Mixing can also involve various filters, such as high pass, low pass, reverb, etc.



Filters



“Easing functions for sounds” (cit. somebody, probably)

Cool audio tricks



- Ducking
- Panning
- Positioning

- Ducking means lowering the volume of certain channels to prioritize something else.
- Common when important sounds or voice overs are played, turn down everything else.
- Can be used to simulate a sudden focus shift ie. One guard yelling and all other guards get their VOs interrupted/turned down to simulate listening.

Cool audio tricks

- Ducking
- Panning
- Positioning

I just found the right
panning joke, but it left
me...

- Panning means shifting sound volume between available speakers.
- In a surround setup that means Forward/Back and Left/Right (maybe even more). But for stereo speakers that means adjusting Left/Right volume.
- You might want to split sounds in a rhythm game, to make them easier to distinguish, simulate hearing loss in your protagonist or other game related choices.
- You can also pan a channel based on position of a source.

Cool audio tricks

- Ducking
- Panning
- Positioning



“I don’t need audio to play this game”
(somebody’s gonna have a bad time)

- 3D: Head-Related Transfer Function, adjust volume and delay between ears based on position and direction of sound emitter.
- 2D: Panning based on screen position, (things on the left are louder in the left speaker, etc.)
- We can also adjust volume of things based how distant they are.
 $\text{volume} = \text{Clamp01}(r - \text{distance})$

Basic sound using SDL

```
#include <SDL3/SDL.h>

// init audio (not strictly necessary, but sometimes audio devices are not handled correctly if not done explicitly)
SDL_Init(SDL_INIT_AUDIO);

SDL_AudioSpec spec_music; // audio information (number of channels, number format of a single sample, sampling frequency)
Uint8 *wav_data = NULL;   // audio data (bytes)
Uint32 wav_data_len = 0;  // in bytes

SDL_LoadWAV("my_audio.wav", &spec_music, &wav_data, &wav_data_len); // loads audio data and metadata (specs, stuff like)

SDL_AudioSpec spec_device; // sound devices have specs like audio files

// creates a representation for the desired audio device (speaker, headset, microphone and so on)
SDL_AudioDeviceID device = SDL_OpenAudioDevice(SDL_AUDIO_DEVICE_DEFAULT_PLAYBACK, &spec_device);
stream = SDL_CreateAudioStream(&spec_music, &spec_device); // runtime information needed to play audio files
SDL_BindAudioStream(device, stream); // tells SDL that the given stream will play on the given device

// combines creating a device, creating a stream, and binding stream to device in one single call. Not really useful for games
// stream = SDL_OpenAudioDeviceStream(SDL_AUDIO_DEVICE_DEFAULT_PLAYBACK, &spec, NULL, NULL);

SDL_ResumeAudioStreamDevice(stream); // plays music (music is paused at creation, so you will need to call this at least once)
SDL_PauseAudioStreamDevice(stream); // pauses music
```

Basic sound using SDL_mixer

```
#include <SDL3_mixer/SDL_mixer.h>

SDL_Init(SDL_INIT_AUDIO); // same as before: technically not necessary, but better safe than sorry
MIX_Init();               // this MUST be called explicitly!

MIX_Mixer* mixer; // handle to our mixer, which wraps around SDL_AudioDevice and adds actual mixing functionality
MIX_Audio* music; // runtime wrapper for music and sound effects

mixer = MIX_CreateMixerDevice(SDL_AUDIO_DEVICE_DEFAULT_PLAYBACK, NULL); // creates SDL_AudioDevice + mixer data
music = MIX_LoadAudio(mixer, "my_audio_in_whatever_format.ogg", false); // load audio and optionally decodes it

// sets the volume of the entire game (0: complete silence, 1: full volume)
// NOTE: going above 1 get very loud VERY fast, beware of possible hardware damage (the computer, or your ears)
MIX_SetMasterGain(mixer, 1.0f);

// plays an audio file without any control (once its started, cannot be paused/muted or anything)
MIX_PlayAudio(mixer, music);
```

Tracks

```
MIX_Track* track = MIX_CreateTrack(mixer);
SDL_PropertiesID track_properties; // array of multiple properties SDL_mixer uses this in a few places
SDL_SetNumberProperty(track_properties, MIX_PROP_PLAY_LOOPS_NUMBER, -1); // set the track to loop infinitely
SDL_SetNumberProperty(track_properties, MIX_PROP_PLAY_FADE_IN_MILLISECONDS_NUMBER, 1000); // fade in on play

SDL_SetAudioIOTrack()
// sets a track playback to 1s from the start
 Sint64 track_pos_frame = MIX_TrackMSToFrames(track, 1000);
MIX_SetTrackPlaybackPosition(track, track_pos_frame);

MIX_PlayTrack(track, track_properties);
MIX_PauseTrack(track);
MIX_StopTrack(track, MIX_TrackMSToFrames(track, 1000)); // stops the track after 1s of

// sets the volume of the entire game (0: complete silence, 1: full volume)
// NOTE: going above 1 get very loud VERY fast, beware of possible hardware damage
// (the computer, or your ears)
MIX_SetTrackGain(track, 1.0f);
```

